- Арифметические операции и операторы сравнения
- Логические операторы (коннективы)
- Условная конструкция. Полная и краткая формы

Программирование в лингвистике

Лекция 2

Логические операции и условия

Преподаватель: Моисеева Анна Юрьевна

Арифметические операции

Скобки работают как в математике!

Операция	Только с int	С float	Со строками
Сложение	1 + 2 = 3 (int)	1.0 + 2 = 3.0 (float)	“a” + “b” = “ab” (str)
Вычитание	3 – 2 = 1 (int)	3.5 – 2.5 = 1.0 (float)	Type Error!
Умножение	2 * 2 = 4 (int)	2.5 * 2 = 5.0 (float)	“a” * 3 = “aaa” (str)
Деление	4 / 2 = 2.0 (float)	5 / 2.5 = 2.0 (float)	Type Error!
Целочисленное деление	9 // 2 = 4 (int)	10 // 7.5 = 1.0 (float)	Type Error!
Остаток от деления	10 % 3 = 1 (int)	1 % 0.5 = 0.0 (float)	Type Error!
Возведение в степень	2 ** 3 = 8 (int)	1.5 ** 2 = 2.25 (float)	Type Error!

А если мы введем то же самое в консоль (с ответом), то результатом будет True.

Операторы сравнения

Возвращают объект типа bool: True (истина) или False (ложь).

Операция	С числами	Со строками
Больше	3 > 1 -> True	“b” > “c” -> False
Меньше	3 < 1 -> False	“a” < “b” -> True
Больше или равно	3 >= 3 -> True	“a” >= “b” -> False
Меньше или равно	1 <= 3 -> True	“a” <= “c” -> True
Равно (не путать с “=“)	3 == 2 -> False	“a” == “A” -> False
Не равно	3 != 2 -> True	“a” != “b” -> True
Используя «и»	1 < 3 and 6 < 3 -> False	“a” in “ab” and “b” in “ab” -> True
Используя «или»	1 < 3 or 6 < 3 -> True	“a” in “c” or “b” in “c” -> False
Используя «не»	1 < 3 and not 6 < 3 -> True	“a” in “ab” and not “b” in “ac” -> True

Скобки тоже работают.

True (1) и False (0) можно оперировать как числами:

True + True = 2

True - False = 1

False * False = 0

Логические операторы

...И особенности их работы

Операнд 1	and / or	Операнд 2	Результат
1 < 3 (True)	and	2 < 3 (True)	True
2 < 3 (True)	and	3 < 1 (False)	False
3 < 1 (False)	and	2 < 3 (True)	False
1 < 3 (True)	or	2 < 3 (True)	True
2 < 3 (True)	or	3 < 1 (False)	True
2 < 1 (False)	or	3 < 1 (False)	False

Условные конструкции

Отступ!
(4 пробела или tab)

elif, else необяза-тельны

Общий шаблон

```
if [условие]:
    [код]
elif [другое условие]:
    [код]
elif [еще условие]:
    [код]
elif (сколько угодно раз):
....
else:
    [код]
```

	Пояснение	Выполняется
if	если	если True
elif (else if)	иначе если	если предыдущее условие False, а это - True
else	иначе	Если все условия выше False

Пример простой проверки:

```
If [что-то]:
    [сделать что-то]
```

```
a = 1
b = 2
if a == b:
  print("1 = 2? Вряд ли")
elif a > b:
  print("1 > 2? Ну не")
elif a < b:
  print("1 < 2? Это верно")
else:
  print("Ну не знаю")
```

```
1 < 2? Это верно
```

Еще примеры

Можно выполнять такие проверки, не прописывая условий (все равно получается bool):

Со строкой

```
a = ""
if a:
  print("a - это не пустая строка")
else:
  print("a - пустая строка")
```

```
a - пустая строка
```

С bool

```
b = False
c = True
if c and b:
  print("b и c являются True")
elif c:
  print("c является True")
# Еще одна проверка
if c and not b:
  print("c является True, a b - False")
```

```
c является True
c является True, a b - False
```

С целым числом

```
[5] x = 5
    c = 0
    if x and c:
      print("x и c не равны 0")
    elif not c:
      print("c равен 0")
```

```
c равен 0
```

Пример красивого кода

```python
user_name = input("Ваше имя: ") # Выбирайте названия объектов Python иллюстративно.
int = int(input()) '''Так делать нельзя. Избегайте совпадений имен ваших
объектов с типами, функциями, методами и др. Python или импортируемых библиотек.'''
user_age = int(input("Ваш возраст: ")) # При присвоении обе стороны отделяются пробелами.
result = (user_age + 5) // 2 ** 3 # Все арифметические операции отбиваются пробелами.
# Усл. констукции, функции, классы, циклы рекомендуется отбивать пустой строкой.

if user_age > 20: # Рекомендуется использовать в качестве отступа не tab, а 4 пробела.
    print(user_name) # Скобки никогда не отбиваются пробелами.

'''Если конструкции, используемые в условном выражении составные, то их следует
занести в скобки:'''
if (user_age > 20) and (user_name == "Вася"):
  print(user_name, user_age) # После запятой ставится пробел.
```

Спасибо за внимание!

Контакты для связи: ajumo@yandex.ru, abyssian03@gmail.com
GitHub: https://github.com/abyssian03/Stud